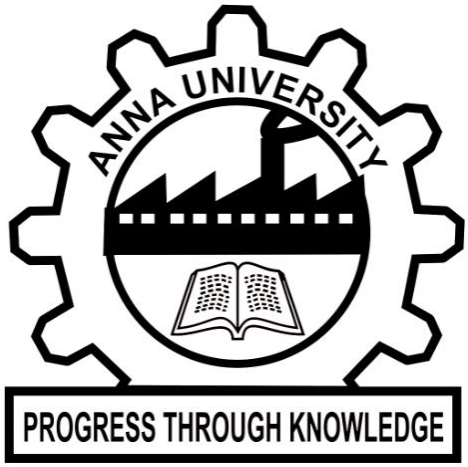
EMPLOYEE MANAGEMENT ANALYSIS USING JAVA FULL STACK

A SUMMER INTERNSHIP REPORT

Submitted by

VIGNESHWARAN S - 810023205011



DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS, ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS, ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that Summer internship project report **EMPLOYEE MANAGEMENT ANALYSIS USING JAVA FULL STACK** is the bonafide work of **VIGNESHWARAN S (810023205011)** who has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.Tech INFORMATION TECHNOLOGY** during the academic year 2026-2027.

FACULTY INCHARGE **HEAD OF THE DEPARTMENT**

ABSTRACT

The internship was successfully completed at **Besant Technologies, Chennai**, from **01 June 2026 to 30 June 2026** under the domain of **Java Full Stack Development**, with training hours from 10:00 AM to 04:00 PM. The internship provided practical exposure to web application development and helped bridge the gap between academic learning and industry practices.

During the training, knowledge was gained in **Java, HTML, CSS, JavaScript and SQL**. These technologies were used to design user interfaces, develop application logic, manage data, and perform CRUD operations. As part of the internship, a project titled **"Employee Management Analysis"** was undertaken to manage employee information through a web-based application. The internship enhanced programming, database management, analytical, and problem-solving skills, providing a strong foundation for future academic projects and professional careers in **Full Stack Java Development**.

TABLE OF CONTENTS

Abstract **6**
CHAPTER 1: INTRODUCTION & OBJECTIVES **7**
1.1 Introduction 7
1.2 Objectives of the Internship 7
1.3 Internship Particulars 7
1.4 Organisation Profile 8
1.5 Technologies Studied 8
1.6 Scope of the Internship 8
CHAPTER 2: SYSTEM STUDY **9**
2.1 Problem Statement Identification 9
2.2 Proposed Solution 9
2.3 Existing System and Limitations 9
2.4 Functional Requirements 10
2.5 Non-Functional Requirements 10
2.6 Feasibility Study 11
2.7 Hardware and Software Requirements 11
2.8 Advantages of the Proposed System 11
CHAPTER 3: ARCHITECTURE **12**
3.1 Architectural Design 12
3.2 Modular Breakdown 13
3.3 Database Design 13
3.4 Data Dictionary 14
3.5 Data Flow 15
3.6 User Roles and Access 16
3.7 Design Principles Followed 17
3.8 Page Navigation Flow 17
CHAPTER 4: DEVELOPMENT AND RESULT ANALYSIS **18**
4.1 System Implementation and Development 18
4.2 Result Analysis and Testing Parameters 22
4.3 Test Cases 22
4.5 Output Screens 23
4.6 Summary of Output Screens 39
4.7 Challenges and Solutions 39
CHAPTER 5: CONCLUSION **40**
5.1 Conclusion 40
5.2 Future Enhancements 40
REFERENCES **40**

LIST OF FIGURES

Figure 3.1: Three-Layer Architecture of the System....12
Figure 3.2: Entity-Relationship Diagram....13
Figure 3.3: Data Flow Diagram – Level 0 (Context Diagram)....15
Figure 3.4: Data Flow Diagram – Level 1....15
Figure 3.5: Use-Case Diagram for Admin and Viewer....16
Figure 3.6: Model-View-Controller Flow....17
Figure 3.7: Page Navigation Flow of the Application....17
Figure 4.1: Project Structure of the Application....18
Figure 4.2: Login Page....23
Figure 4.3: Error Message for Invalid Login....24
Figure 4.4: Dashboard of the Application....25
Figure 4.5: Employee List Page....26
Figure 4.6: Add Employee Form....27
Figure 4.7: Client-Side Validation Messages....28
Figure 4.8: Duplicate Employee ID Rejected....29
Figure 4.9: Success Message after Adding an Employee....30
Figure 4.10: Search Result for the Development Department....31
Figure 4.11: No Records Found Message....32
Figure 4.12: Edit Employee Form....33
Figure 4.13: Success Message after Updating an Employee....34
Figure 4.14: Confirmation Dialog before Deleting a Record....35
Figure 4.15: Department-wise Employee Report....36
Figure 4.16: Employee List for a Viewer....37
Figure 4.17: Access Denied Message for a Restricted Operation....38

CHAPTER 1: INTRODUCTION & OBJECTIVES

1.1 Introduction

A full stack web application has three parts that work together: a front end that the user sees, a back end that applies the rules of the application, and a database that keeps the data. Full Stack Java Development is the practice of building all three parts with Java as the main language of the back end.

On the front end, HTML gives a page its structure, CSS controls its appearance and JavaScript makes it respond to the user, for example by checking a form before it is sent. On the back end, Java receives each request, checks it, applies the business rules and talks to the data store. Structured Query Language (SQL) is the standard way of creating and querying the tables of a relational database, where keys and constraints keep the data consistent.

During the internship at Besant Technologies these ideas were applied to a small but complete project, Employee Management Analysis, which manages employee records through Create, Read, Update and Delete (CRUD) operations.

1.2 Objectives of the Internship

- To understand how a web application is built from the browser to the data store.
- To learn Java, HTML, CSS, JavaScript and SQL and use them together in one project.
- To implement CRUD operations with input validation and role-based access.
- To test the application with planned test cases and record the results.
- To develop problem-solving, time management and documentation skills expected in industry.

1.3 Internship Particulars

The details below are taken from the offer letter, internship letter and certificate of attendance issued by the organisation.

Particular	Details
Name of the Student	VIGNESHWARAN S
Register Number	810023205011
Programme	B.Tech Information Technology (Seventh semester, course IT3711)
Institution	University College of Engineering, BIT Campus, Anna University, Tiruchirappalli – 620 024
Organisation	Besant Technologies, Chennai
Domain	Java Full Stack Development
Project Title	Employee Management Analysis
Duration	01 June 2026 to 30 June 2026 (1 Month)
Training Time	10:00 AM to 04:00 PM
Role	Project Trainee

1.4 Organisation Profile

Besant Technologies is a technology training organisation with its Chennai centre at No. 119, 11th Main Road, Vijaya Nagar, Velachery, Chennai, Tamil Nadu – 600042. It offers instructor-led training and internship programmes in software technologies to engineering students.

For this internship the organisation issued three documents: an **Offer Letter** dated 10 May 2026 for the Java Full Stack internship from 01 June 2026 to 30 June 2026; an **Internship Letter** (July 2026) confirming that the internship was completed successfully; and a **Certificate of Attendance** (July 2026) confirming that the project titled "Employee Management Analysis" was carried out under the organisation's guidance. The letters were issued by the Manager – Human Resources.

1.5 Technologies Studied

Java is a general-purpose, object-oriented language. Classes, objects, encapsulation, collections and exception handling are the basis of the back end. Java also provides a built-in HTTP server and the JDBC interface for databases.

HTML defines the structure of a page: headings, tables, forms, input fields and buttons. **CSS** controls colours, fonts, spacing and layout so that every page of the application looks consistent.

JavaScript adds behaviour in the browser. In this project it validates the form before submission, shows messages next to the wrong fields and opens the delete-confirmation dialog.

SQL is used with a relational database to create tables, insert, read, update and delete rows, and to produce summaries with GROUP BY, COUNT and JOIN. Primary keys, foreign keys and UNIQUE constraints protect the integrity of the data.

1.6 Scope of the Internship

- Learning the basics of Java and object-oriented programming.
- Designing web pages with HTML and CSS and adding behaviour with JavaScript.
- Understanding the request–response cycle between the browser and the server.
- Designing a relational database and writing SQL for CRUD operations.
- Analysing requirements, designing the architecture and modules of the employee management system.
- Building the application, testing it and documenting the work in this report.

CHAPTER 2: SYSTEM STUDY

2.1 Problem Statement Identification

Organisations keep a large amount of employee information that must be stored, updated and found quickly. When the information is spread across registers and separate spreadsheet files, the same record is repeated in many places, copies disagree with each other, searching is slow and summaries such as the number of employees in each department take a long time to prepare.

A web-based employee management system solves these problems by keeping every record in one place, checking data when it is entered and showing the right information to the right user.

2.2 Proposed Solution

The proposed system, **Employee Management Analysis**, is a web application built with the Java Full Stack approach. HTML, CSS and JavaScript provide the interface, Java handles the logic and a data-access class handles storage. Each employee has an ID, name, email, phone number, department, designation, joining date and status.

Users log in as Admin or Viewer. An Admin can add, view, search, update and delete records and see the department report. A Viewer can only log in, view, search and read the report. Every input is validated twice: in the browser for convenience and on the server for safety.

2.3 Existing System and Limitations

In many small organisations employee details are kept in registers or spreadsheets maintained by different people. The main limitations of this manual method are:

- **Redundancy:** the same employee appears in several files.
- **Inconsistency:** a change made in one copy is missing from the others.
- **Slow retrieval:** finding one employee means opening many files or pages.
- **Human errors:** spelling mistakes, wrong dates and missing fields during typing.
- **Poor security:** anyone with the file can copy or change it.
- **Difficult reporting:** department-wise and status-wise summaries take time.
- **No validation:** nothing stops duplicate or invalid entries.

2.4 Functional Requirements

The functional requirements describe what the system must do.

No.	Requirement	Description
FR1	User Login	Only registered users can log in with a username and password.
FR2	Add Employee	An Admin can add a new employee record with all required details.
FR3	View Employees	The system lists employees in a table with the important details.
FR4	Search Employee	Employees can be searched by ID, name or department.
FR5	Update Employee	An Admin can modify the details of an existing employee.
FR6	Delete Employee	An Admin can delete a record after confirming the action.
FR7	Validation	Mandatory fields, formats and duplicate IDs are checked before saving.
FR8	Reports	A summary shows the number of employees in each department.

2.5 Non-Functional Requirements

- **Usability:** simple pages with clear labels, easy for a new user.
- **Performance:** search and update respond within a short time.
- **Reliability:** an error must not lose or corrupt stored data.
- **Security:** only authorised users can see or change data; roles are checked on the server.
- **Maintainability:** code is divided into model, data-access, controller and view parts.
- **Portability:** the application runs on any operating system that has Java and a web browser.

2.6 Feasibility Study

Technical feasibility: Java, HTML, CSS, JavaScript and SQL are open and well documented, and the application needs only an ordinary computer. **Economic feasibility:** all tools used are free, so the cost of development is very low. **Operational feasibility:** the interface is simple enough for a user with basic computer skills to operate with little training.

2.7 Hardware and Software Requirements

Component	Requirement
Processor	Intel Core i3 or higher (or equivalent)
RAM	4 GB minimum (8 GB recommended)
Hard Disk	5 GB of free space
Operating System	Windows 10 / 11, Linux or macOS
Language and runtime	Java Development Kit (JDK) 17 or higher
Front end	HTML5, CSS3 and JavaScript
Data storage	Prototype: CSV file; database version: relational database with SQL
Web browser	Google Chrome, Microsoft Edge or Mozilla Firefox

2.8 Advantages of the Proposed System

- All employee data is in one place, so duplicates and conflicting copies are avoided.
- A record can be found in seconds by ID, name or department.
- Changes are visible to every user immediately.
- Login and roles control who can view or change data.
- The department report is produced instantly and accurately.
- The modular design makes the application easy to extend.

CHAPTER 3: ARCHITECTURE

3.1 Architectural Design

The system follows a three-layer architecture. The **Presentation Layer** (HTML, CSS and JavaScript) runs in the browser. The **Application Layer** (Java) receives requests, validates them, applies the role rules and calls the data-access class. The **Data Layer** holds the employee, department and user data. Each layer talks only to the layer next to it, as shown in Figure 3.1.

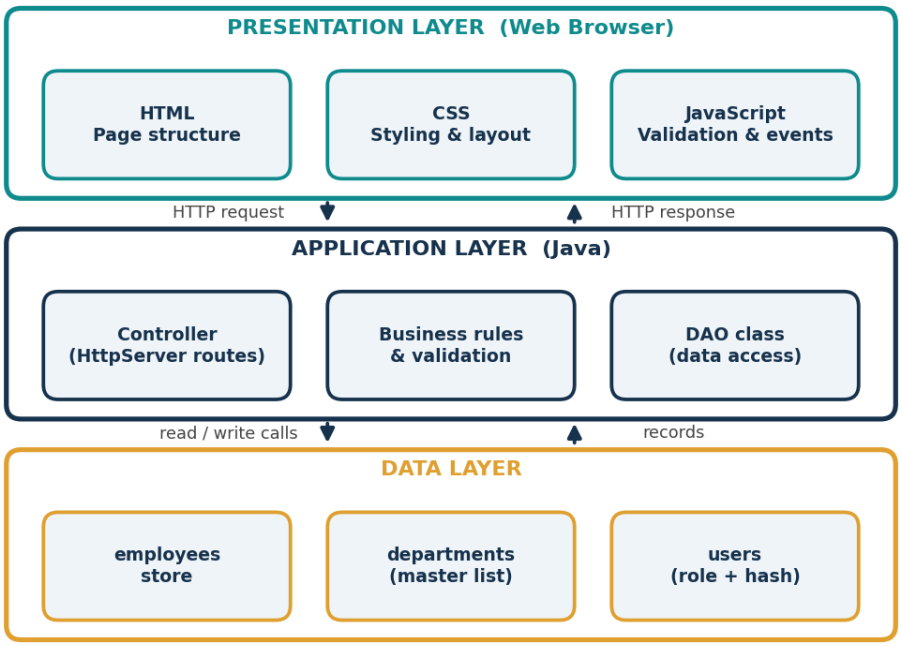


Figure 3.1: Three-Layer Architecture of the System

Working of the architecture

- The user opens the application in a web browser and logs in.
- The user fills a form or chooses an action on a page; JavaScript checks the values.
- The request reaches the Java application, which validates it again and checks the user's role.
- The data-access class reads or writes the employee records.
- The result is turned into an HTML page and returned to the browser.

3.2 Modular Breakdown

The application is divided into five modules. Each module does one job, and together they provide the complete system.

Module	Functions
1. User Authentication	Login, password check against a stored hash, session handling, logout.
2. Employee Management	Add an employee, view the list, search by ID, name or department.
3. Employee Update	Open a record in the edit form and save the changed details.
4. Employee Deletion	Delete a record after a confirmation dialog; viewers are blocked.
5. Data Management and Reports	Read and write records through the data-access class; department-wise summary.

3.3 Database Design

The data is designed with the relational model using three entities: **Employee**, **Department** and **User**. Each employee belongs to exactly one department, so department_id in the Employee table is a foreign key to the Department table. The User table stores login accounts and is independent of the other two. Figure 3.2 shows the entity-relationship diagram.

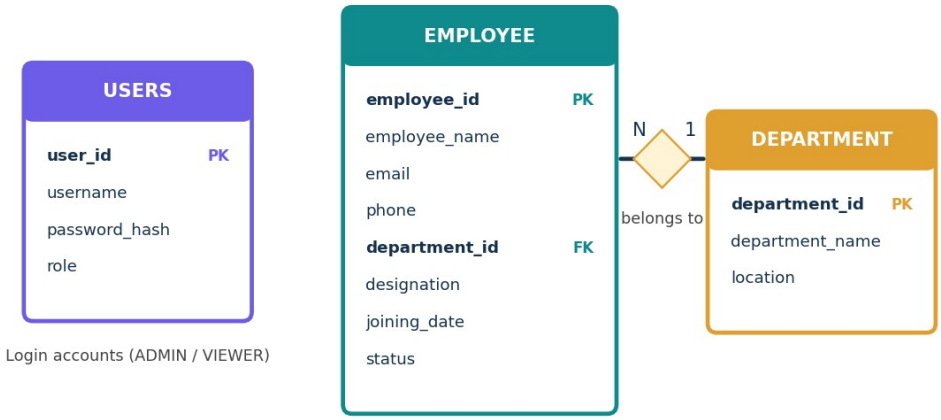


Figure 3.2: Entity-Relationship Diagram

3.4 Data Dictionary

Table: employee

Field	Data Type	Constraint	Description
employee_id	INT	PRIMARY KEY	Unique number of the employee
employee_name	VARCHAR(50)	NOT NULL	Full name
email	VARCHAR(80)	UNIQUE, NOT NULL	Official email address
phone	VARCHAR(15)	NOT NULL	10-digit contact number
department_id	INT	FOREIGN KEY	Department of the employee
designation	VARCHAR(50)	NOT NULL	Job title
joining_date	DATE	NOT NULL	Date of joining
status	VARCHAR(10)	DEFAULT 'ACTIVE'	ACTIVE or INACTIVE

Table: department

Field	Data Type	Constraint	Description
department_id	INT	PRIMARY KEY	Unique number of the department
department_name	VARCHAR(50)	UNIQUE, NOT NULL	Name of the department
location	VARCHAR(50)	—	Place where it works

Table: users

Field	Data Type	Constraint	Description
user_id	INT	PRIMARY KEY	Unique number of the user
username	VARCHAR(30)	UNIQUE, NOT NULL	Login name
password_hash	VARCHAR(64)	NOT NULL	SHA-256 hash of the password
role	VARCHAR(15)	NOT NULL	ADMIN or VIEWER

3.5 Data Flow

At the highest level (Figure 3.3) the whole application is one process that exchanges data with the user and the data store. Figure 3.4 splits it into four processes: Authenticate User (1.0) reads the users data; Manage Employees (2.0) reads and writes employee data; Search Employees (3.0) reads employee data; and Generate Reports (4.0) reads employee and department data.



Figure 3.3: Data Flow Diagram – Level 0 (Context Diagram)

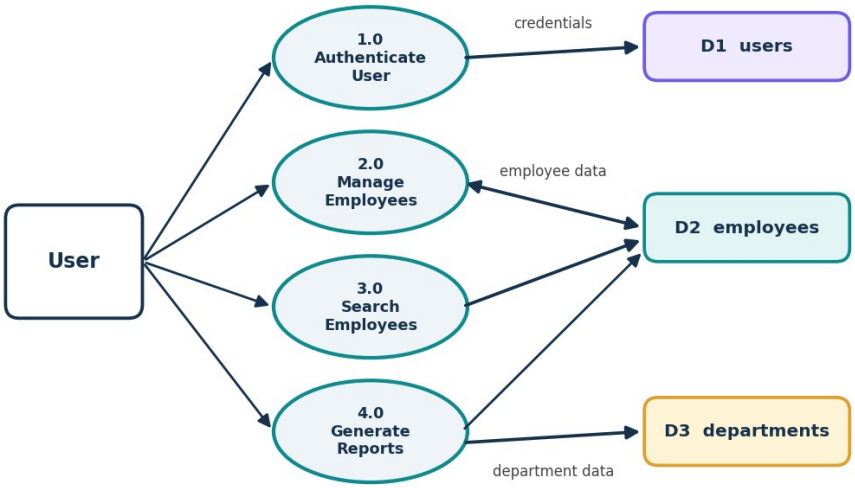


Figure 3.4: Data Flow Diagram – Level 1

3.6 User Roles and Access

Each role may perform only the operations it needs. Access is checked in the Java application and not only in the page, because a hidden button can still be requested by typing its address.

Operation	Admin	Viewer	Remarks
Login / Logout	Yes	Yes	Valid credentials required
View employee list	Yes	Yes	After login only
Search employee	Yes	Yes	By ID, name or department
View department report	Yes	Yes	Read-only summary
Add employee	Yes	No	Validated before saving
Update employee	Yes	No	ID cannot be changed
Delete employee	Yes	No	Confirmation required

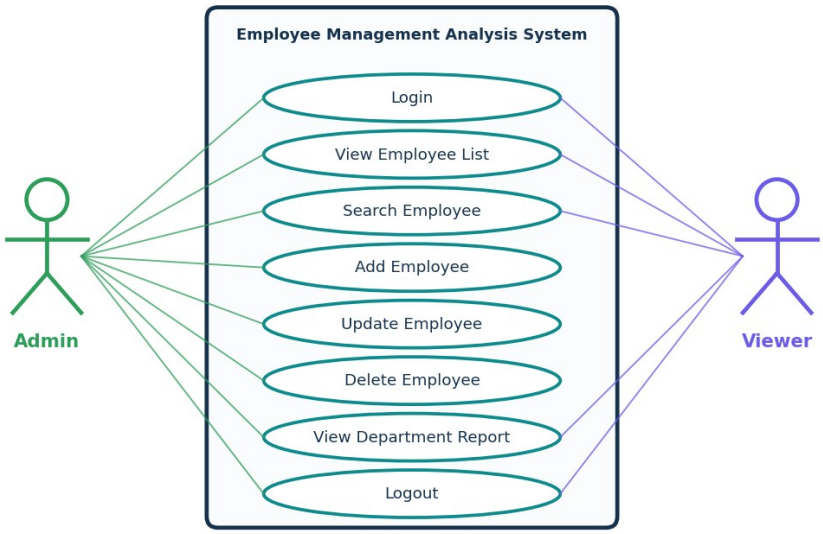


Figure 3.5: Use-Case Diagram for Admin and Viewer

3.7 Design Principles Followed

- **Separation of concerns:** pages, logic and data access are kept apart.
- **Model–View–Controller:** the controller handles each request, calls the model and selects the view (Figure 3.6).
- **Single responsibility:** each method does one clearly defined task.
- **Validation at both ends:** the browser checks for convenience, the server checks for safety.
- **Least privilege:** a user gets only the access needed for the work.

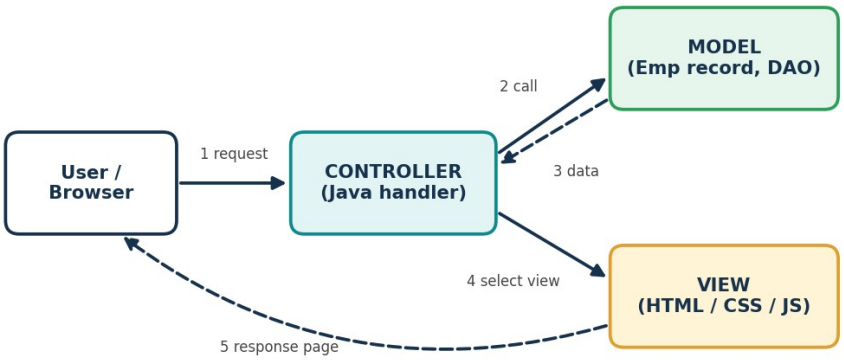


Figure 3.6: Model-View-Controller Flow

3.8 Page Navigation Flow

After login the user reaches the dashboard, from which the employee list, the add form and the report are opened. Edit, delete and search start from the employee list (Figure 3.7).

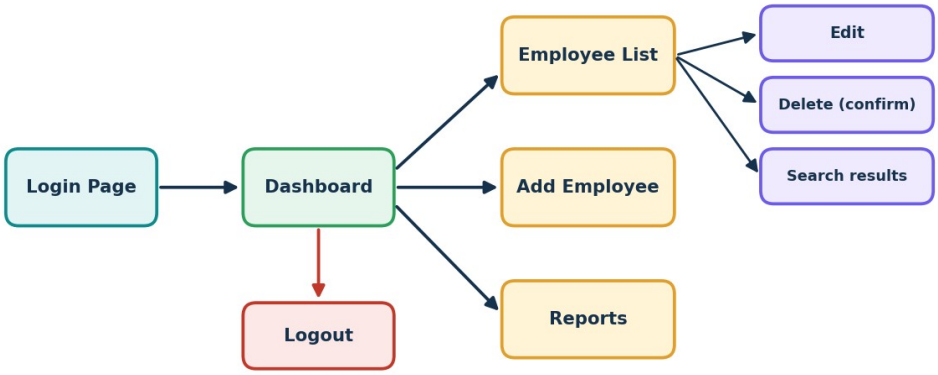


Figure 3.7: Page Navigation Flow of the Application

CHAPTER 4: DEVELOPMENT AND RESULT ANALYSIS

4.1 System Implementation and Development

The application was implemented as a Java program that starts its own web server on port 8080 and serves all pages under the path /EmployeeManagement. HTML and CSS build the pages, JavaScript validates forms and opens the delete dialog, and the Java code contains the controller, the model and the data-access methods. Passwords are stored as SHA-256 hashes, and every value shown on a page is HTML-escaped so that characters such as quotes and angle brackets are displayed as text and cannot run as code.

Development environment

Purpose	Tool / Technology
Programming language	Java (JDK 21; JDK 17 or higher is required)
Web server	Built-in Java HTTP server (com.sun.net.httpserver)
Web pages	HTML5, CSS3, JavaScript
Data storage	CSV file written by the data-access methods
Testing	Web browser with planned manual test cases
Documentation	Microsoft Word

Project structure

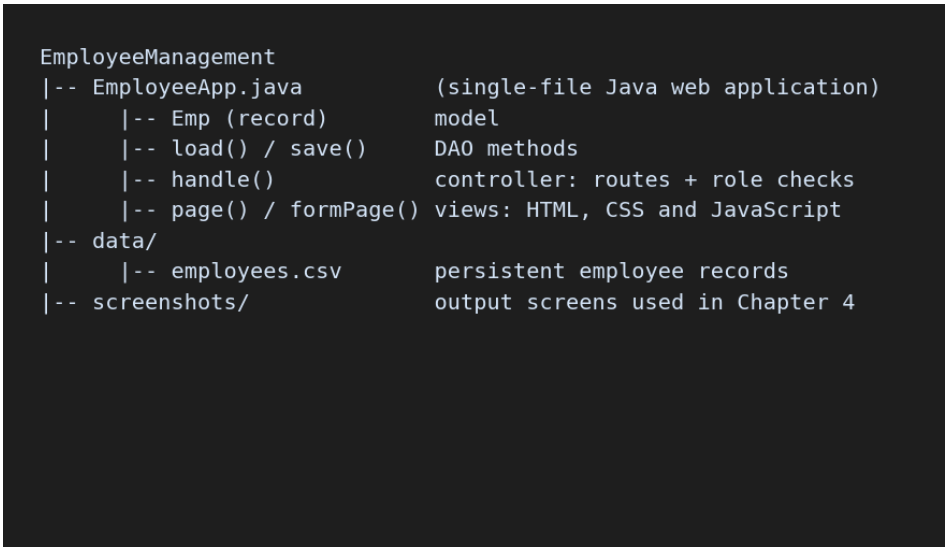


Figure 4.1: Project Structure of the Application

Front-End Implementation

The add and edit pages share one form. The form is marked novalidate so that the JavaScript below, rather than the browser, reports the errors next to each field. The structure of the form is shown first.

```
<form id="ef" method="post" action="/EmployeeManagement/add" novalidate>
  <label>Employee ID *</label>      <input name="id" id="id">
  <label>Employee Name *</label>    <input name="name" id="name">
  <label>Email Address *</label>    <input name="email" id="email">
  <label>Phone Number *</label>     <input name="phone" id="phone">
  <label>Department *</label>
  <select name="dept"> <option>Development</option> <option>Testing</option>
                       <option>Support</option> <option>Human Resources</option> </select>
  <label>Designation *</label>      <input name="desig" id="desig">
  <label>Joining Date *</label>     <input name="joined" id="joined" type="date">
  <button>Save Employee</button> <a href="/EmployeeManagement/employees">Cancel</a>
</form>
```

The validation function checks every field when the form is submitted. If any value is wrong, the error text is shown below the field, the field is highlighted and the form is not sent.

```
document.getElementById('ef').addEventListener('submit', function (ev) {
  var bad = [];
  function chk(id, ok, msg) {
    var i = document.getElementById(id), e = document.getElementById('e_' + id);
    if (!ok) { i.classList.add('err'); e.textContent = msg; bad.push(msg); }
    else     { i.classList.remove('err'); e.textContent = ''; }
  }
  chk('id',    /^[0-9]+$/.test(id.value.trim()), 'Enter a numeric employee ID');
  chk('name',  name.value.trim() !== '',         'Employee name is required');
  chk('email', /^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(email.value.trim()),
                                                 'Enter a valid email address');
  chk('phone', /^[0-9]{10}$/.test(phone.value.trim()), 'Enter a 10-digit phone number');
  chk('desig', desig.value.trim() !== '',        'Designation is required');
  chk('joined', joined.value !== '',             'Joining date is required');
  if (bad.length) { ev.preventDefault(); /* show the summary banner */ }
});
```

Back-End Implementation

A record class represents an employee, and two data-access methods read and write the employee file. Keeping these methods separate from the page code means the storage can later be changed to a database without touching the pages.

```
record Emp(int id, String name, String email, String phone,
           String dept, String desig, String joined, String status) {}

static synchronized List<Emp> load() throws IOException {
    List<Emp> l = new ArrayList<>();
    if (!Files.exists(DATA)) return l;
    for (String line : Files.readAllLines(DATA)) {
        if (line.isBlank()) continue;
        String[] p = line.split(",", -1);
        l.add(new Emp(Integer.parseInt(p[0]), p[1], p[2], p[3], p[4], p[5], p[6], p[7]));
    }
    l.sort(Comparator.comparingInt(Emp::id));
    return l;
}
static synchronized void save(List<Emp> l) throws IOException {
    List<String> out = new ArrayList<>();
    for (Emp e : l) out.add(String.join(",", "" + e.id(), e.name(), e.email(),
            e.phone(), e.dept(), e.desig(), e.joined(), e.status()));
    Files.write(DATA, out);
}
```

The server-side validation repeats the checks of the browser and also rejects a duplicate employee ID or email. The role check comes before any restricted operation, so a viewer who types the address of the delete page directly still receives an Access Denied response.

```
if (!id.matches("\\d+")) fe.put("id", "Enter a numeric employee ID");
else if (!edit && all.stream().anyMatch(e -> e.id() == Integer.parseInt(id)))
    fe.put("id", "Duplicate employee ID");
if (p.getOrDefault("name", "").trim().isEmpty()) fe.put("name", "Employee name is required");
if (!em.matches("[^@\\s]+@[^@\\s]+\\.[^@\\s]+")) fe.put("email", "Enter a valid email
address");

// role check in the controller (runs on every request)
boolean admin = u[1].equals("ADMIN");
if ((path.equals("/add") || path.equals("/edit") || path.equals("/delete")) && !admin) {
    send(ex, 403, accessDeniedPage());   // no change is made to the data
    return;
}
```

SQL for the database version

The prototype stores its records in a file. The relational design of Section 3.3 is the schema intended for the database version, in which the same data-access methods would use JDBC and prepared statements. The statements below create that schema and perform the CRUD operations and reports.

```
CREATE TABLE department (
  department_id   INT PRIMARY KEY,
  department_name VARCHAR(50) UNIQUE NOT NULL,
  location        VARCHAR(50));

CREATE TABLE employee (
  employee_id   INT PRIMARY KEY,
  employee_name VARCHAR(50) NOT NULL,
  email         VARCHAR(80) UNIQUE NOT NULL,
  phone         VARCHAR(15) NOT NULL,
  department_id INT REFERENCES department(department_id),
  designation   VARCHAR(50) NOT NULL,
  joining_date  DATE NOT NULL,
  status        VARCHAR(10) DEFAULT 'ACTIVE');

-- Create / Read / Update / Delete
INSERT INTO employee VALUES (11,'Lavanya K','lavanya@example.com','9876543210',2,'QA
Engineer','2026-06-01','ACTIVE');
SELECT * FROM employee WHERE department_id = 1 ORDER BY employee_id;
UPDATE employee SET designation = 'Senior Developer' WHERE employee_id = 1;
DELETE FROM employee WHERE employee_id = 8;

-- Reports
SELECT department_id, COUNT(*) AS total_employees FROM employee GROUP BY department_id;
SELECT e.employee_name, d.department_name
FROM employee e INNER JOIN department d ON e.department_id = d.department_id;
```

4.2 Result Analysis and Testing Parameters

The application was tested for login, validation, CRUD operations, search, report and access control. Each test was run on the running application, and the output screens in Section 4.5 are the screenshots taken during these tests. The testing parameters were: login and authentication, validation of the employee form, CRUD operations, search and filtering, department-wise report, role-based access, and the handling of special characters.

4.3 Test Cases

ID	Test Case	Input	Expected Result	Result
TC-01	Valid login	admin / admin123	Dashboard is shown	Pass
TC-02	Invalid login	Wrong password	Error message on login page	Pass
TC-03	Add employee	All fields valid	Saved and shown in list	Pass
TC-04	Empty name	Name left blank	Form rejected with message	Pass
TC-05	Duplicate ID	Existing ID 5	Duplicate rejected	Pass
TC-06	Invalid email	Email without @	Validation message	Pass
TC-07	Search employee	Department: Development	Four matching records	Pass
TC-08	Search unknown	ID 99	"No records found"	Pass
TC-09	Update employee	New designation	New value saved	Pass
TC-10	Delete employee	Delete with confirmation	Record removed from list	Pass
TC-11	Special characters	Quotes and <b> tag in name	Stored and shown as plain text	Pass
TC-12	Viewer restriction	Viewer opens delete address	403 Access Denied, no change	Pass

All twelve test cases gave the expected result. The analysis shows that validation at both ends, role checks on the server and the escaping of displayed values together keep the data accurate and the application safe from the common mistakes of a first project.

4.5 Output Screens

This section shows the output screens of the running application in the order a user normally works with it, from the login page to the access-control screen.

4.5.1 Login Page

The login page is the entry point of the application. The user enters a username and password; the password is masked while typing and is compared with the stored hash. Only registered users reach the dashboard.

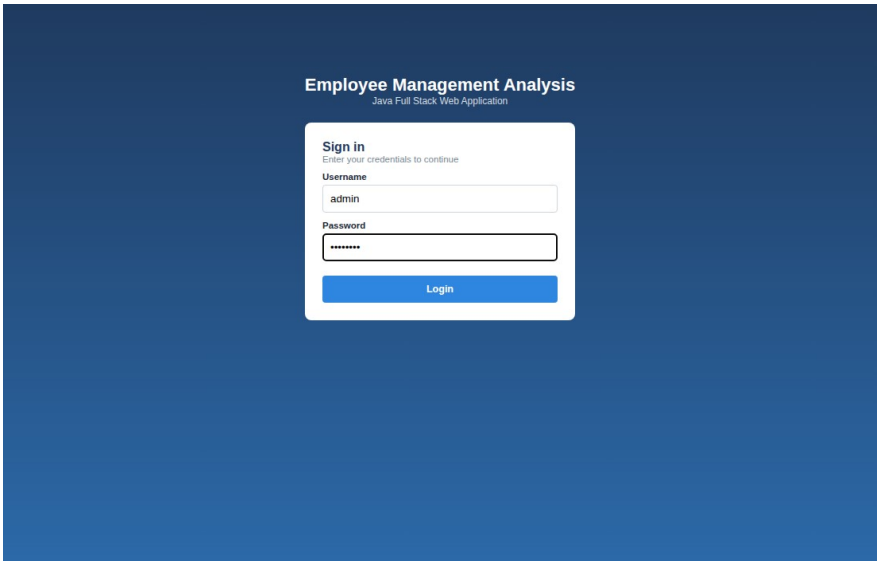


Figure 4.2: Login Page

4.5.2 Login Validation

When the password is wrong the user stays on the login page, the password field is highlighted and a clear message is shown without technical details (test case TC-02).

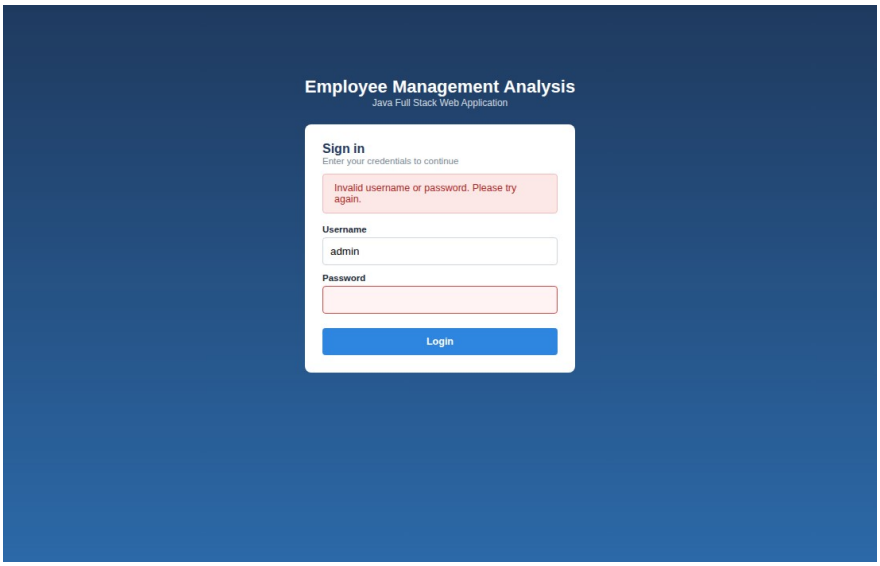


Figure 4.3: Error Message for Invalid Login

4.5.3 Dashboard

After login the dashboard shows the total, active and inactive employees and the number of departments, a bar chart of employees in each department and quick-action buttons.

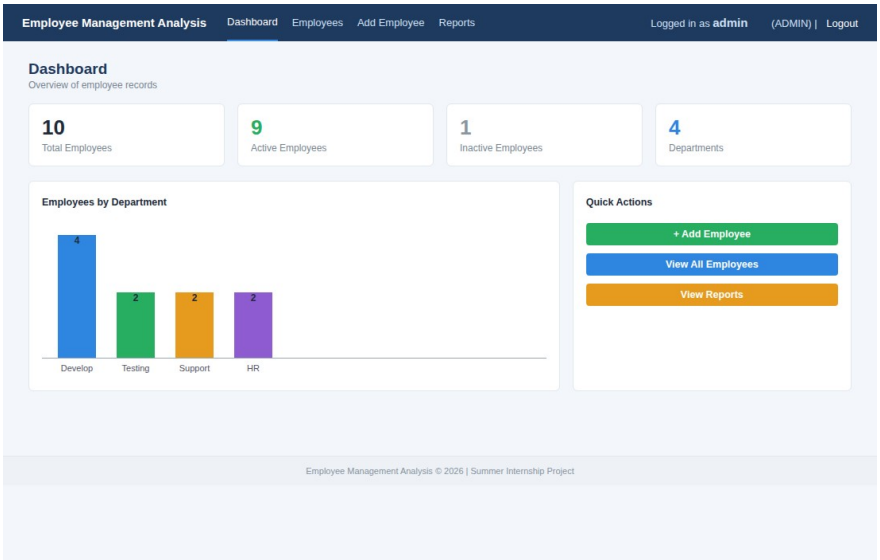


Figure 4.4: Dashboard of the Application

4.5.4 Employee List

The list shows every record with its ID, name, email, department, designation, joining date and a coloured status label. The search box, department filter, Add button and Edit and Delete buttons are on this page.

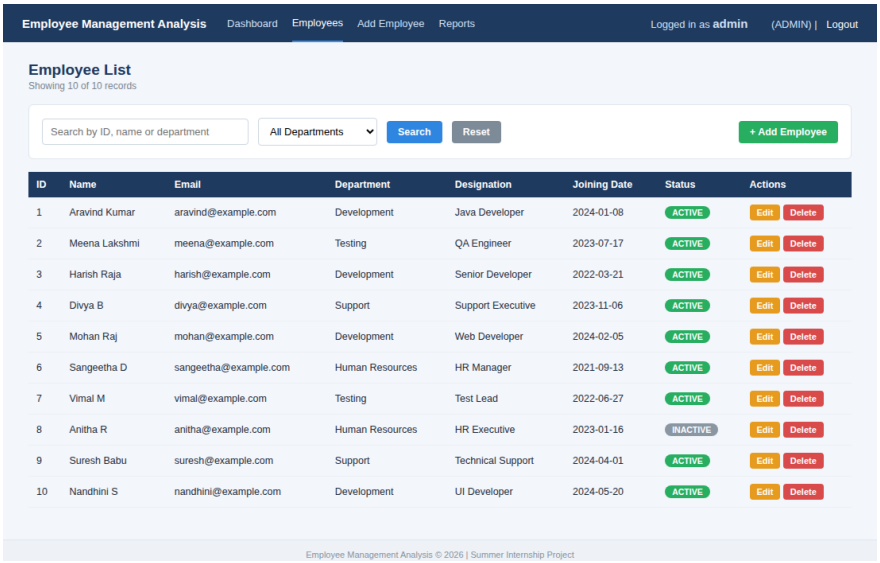


Figure 4.5: Employee List Page

4.5.5 Add Employee Form

The form collects the details of a new employee. Fields marked with an asterisk are mandatory, the department is chosen from a list and the joining date uses a date picker.

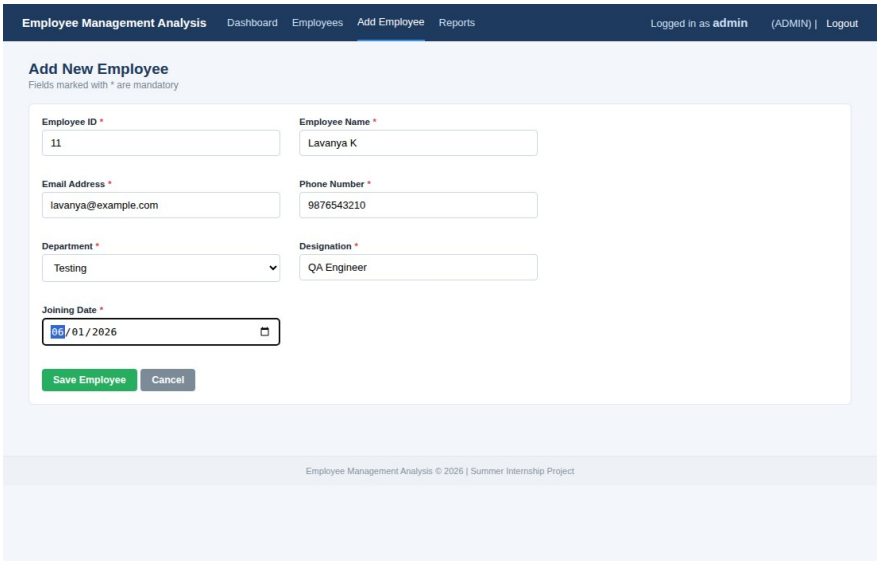


Figure 4.6: Add Employee Form

4.5.6 Form Validation

JavaScript checks the form before it is submitted. Here the name is blank and the email has no @ symbol, so both fields are highlighted and a message appears below each of them (test cases TC-04 and TC-06).

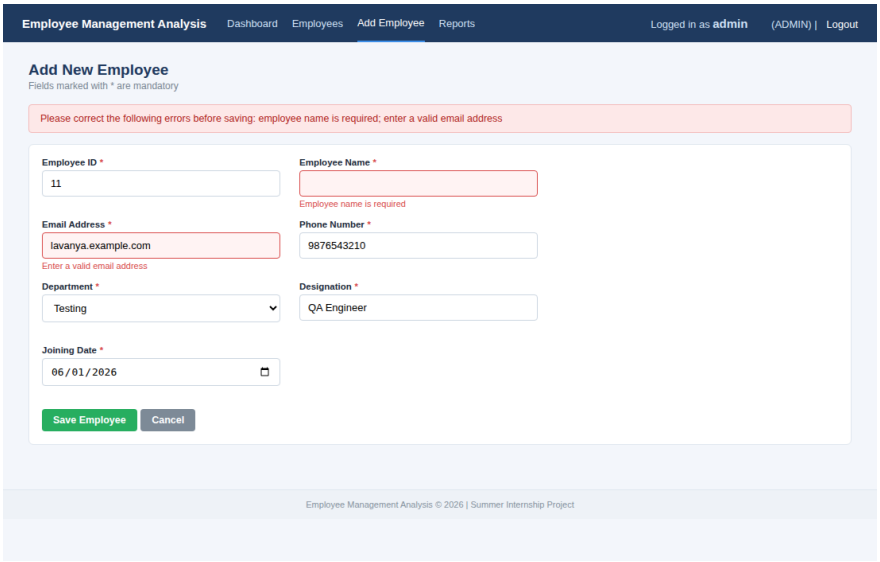


Figure 4.7: Client-Side Validation Messages

4.5.7 Duplicate Employee ID

Employee IDs must be unique. When an existing ID is entered, the server rejects the record and shows a simple message instead of a technical error (test case TC-05).

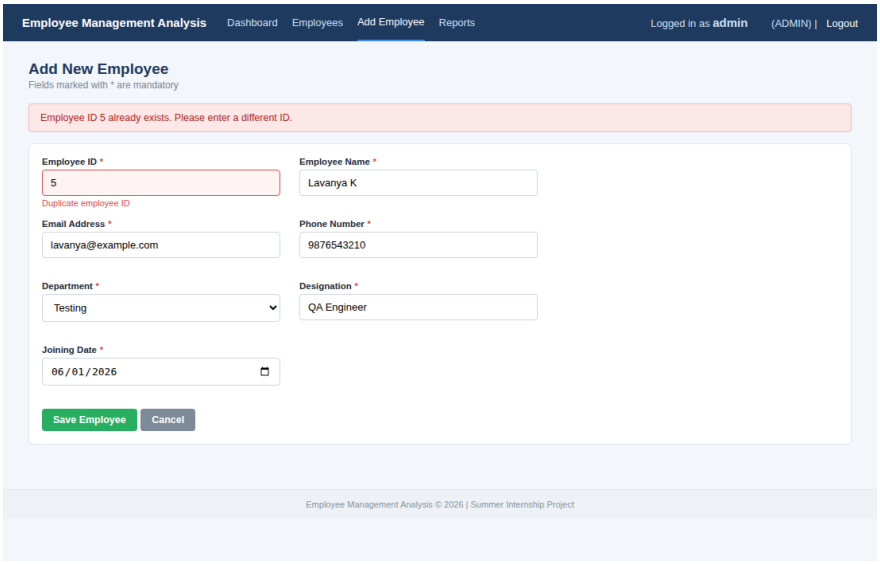


Figure 4.8: Duplicate Employee ID Rejected

4.5.8 Record Saved Successfully

When all values are valid the record is saved and the list is shown with a success message. The new employee is highlighted so that it is easy to find (test case TC-03).

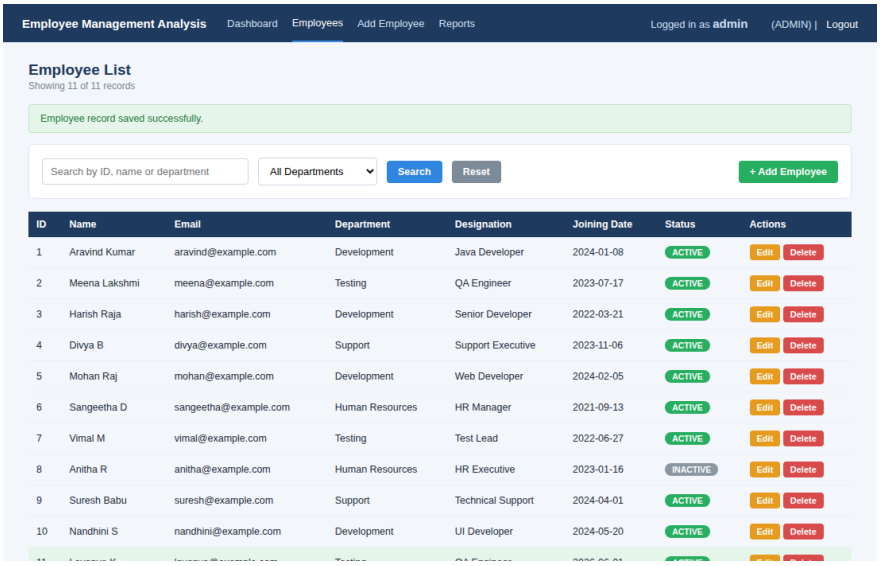


Figure 4.9: Success Message after Adding an Employee

4.5.9 Search by Department

Employees can be searched by ID, name or department. Here the Development department is selected and only its four employees are shown with the number of records found (test case TC-07).

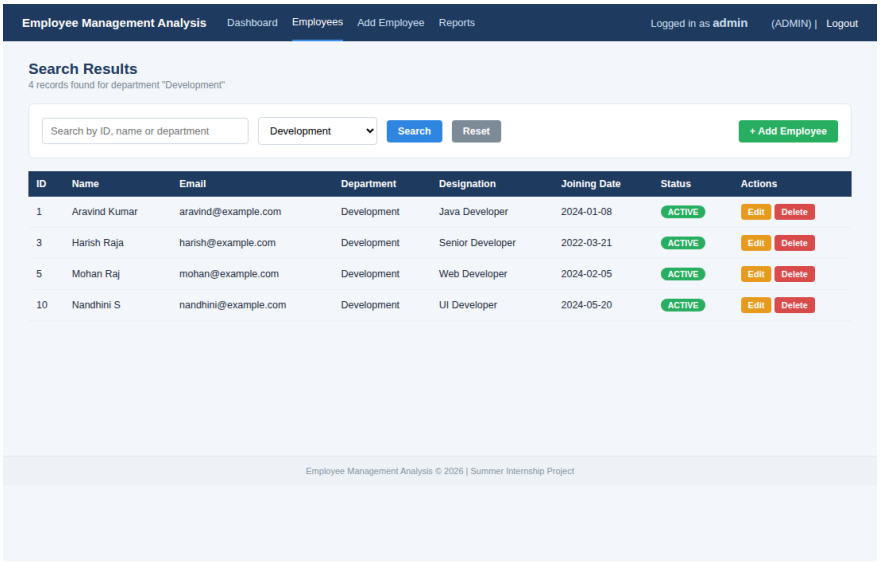


Figure 4.10: Search Result for the Development Department

4.5.10 Search with No Result

When nothing matches the search, the system shows a clear "No records found" message instead of an empty table (test case TC-08).

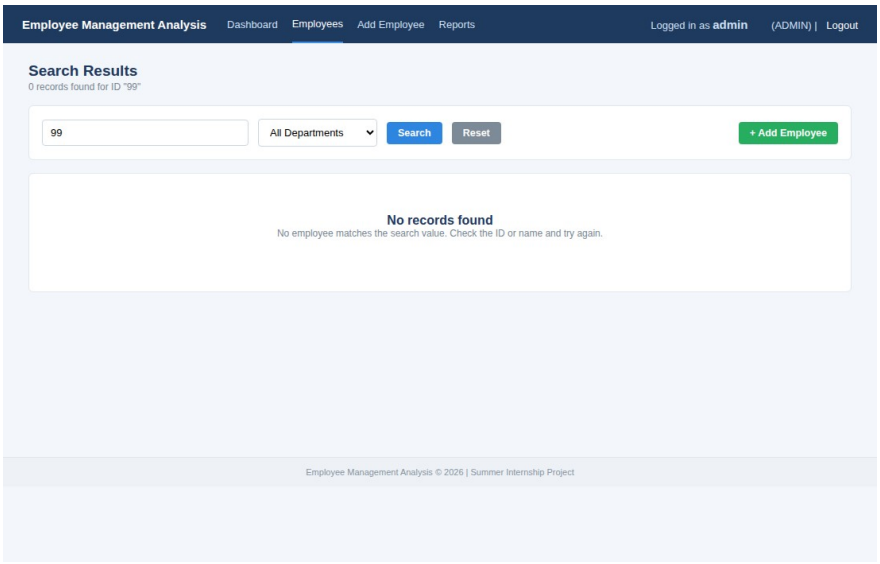


Figure 4.11: No Records Found Message

4.5.11 Edit Employee

The edit form opens with the existing details filled in. The employee ID is read-only because it is the key of the record. Here the designation is being changed to Senior Developer.

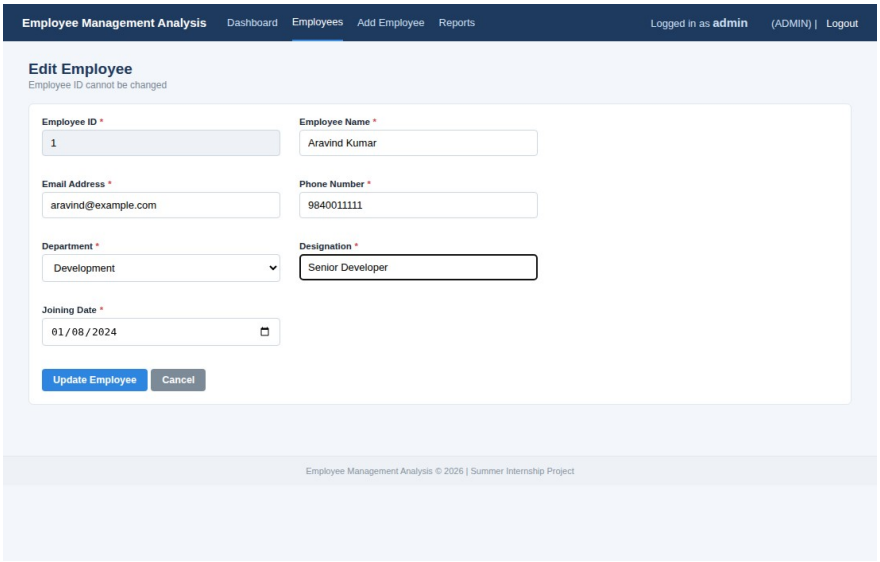


Figure 4.12: Edit Employee Form

4.5.12 Update Confirmation

After the update is saved the list is shown again with a success message and the changed record is highlighted (test case TC-09).

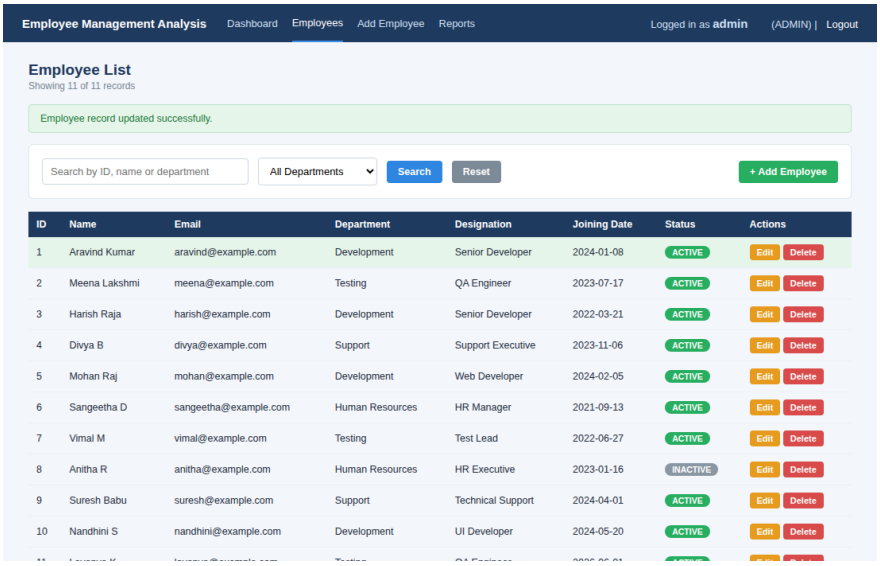


Figure 4.13: Success Message after Updating an Employee

4.5.13 Delete Confirmation

Deleting a record cannot be undone, so a dialog shows the ID, name, department and designation and asks for confirmation. The record is removed only when the user chooses Yes, Delete; Cancel closes the dialog (test case TC-10).

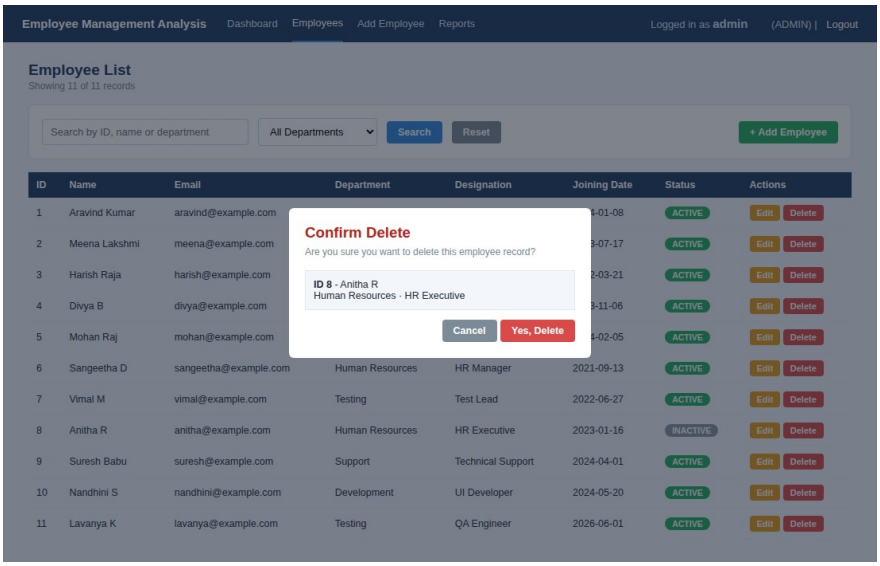


Figure 4.14: Confirmation Dialog before Deleting a Record

4.5.14 Department-wise Report

The report shows the total, active and inactive employees of each department with overall totals, and a bar chart of the same distribution. It meets requirement FR8.

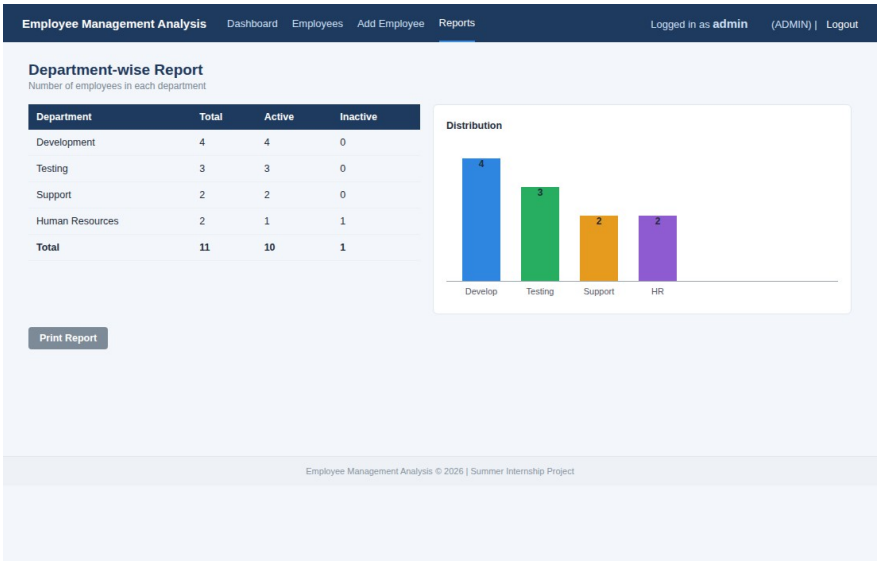


Figure 4.15: Department-wise Employee Report

4.5.15 Viewer Role Screen

A viewer can log in, see the list and search, but the Add, Edit and Delete options are not shown. This applies the least-privilege principle of Section 3.7.

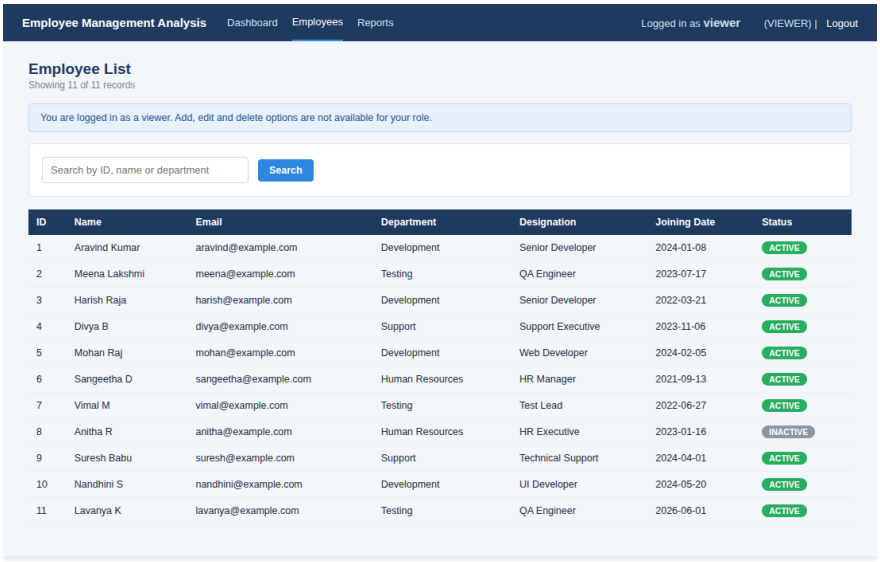


Figure 4.16: Employee List for a Viewer

4.5.16 Access Denied

Hiding buttons is not enough, because a page address can be typed directly. The server checks the role again and returns an Access Denied page for restricted operations, with no change to the data (test case TC-12).

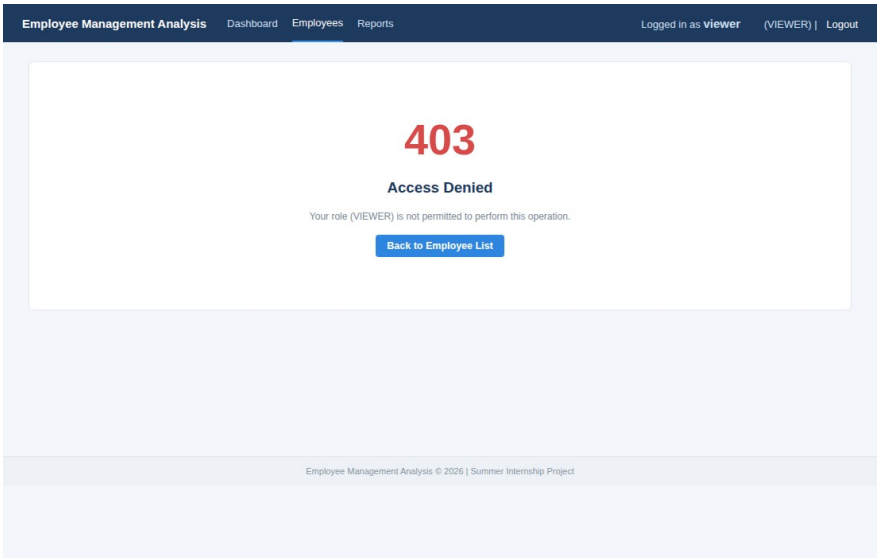


Figure 4.17: Access Denied Message for a Restricted Operation

4.6 Summary of Output Screens

Figure No.	Screen	Requirement / Test Case
4.2, 4.3	Login and invalid login	FR1; TC-01, TC-02
4.4	Dashboard	FR3, FR8
4.5	Employee list	FR3
4.6	Add employee form	FR2
4.7, 4.8	Validation and duplicate ID	FR7; TC-04, TC-05, TC-06
4.9	Record saved	FR2; TC-03
4.10, 4.11	Search and no result	FR4; TC-07, TC-08
4.12, 4.13	Edit and update	FR5; TC-09
4.14	Delete confirmation	FR6; TC-10
4.15	Department-wise report	FR8
4.16, 4.17	Viewer role and access denied	Section 3.6; TC-12

4.7 Challenges and Solutions

Challenge	Solution
Connecting the browser, Java code and data store	Traced one operation from the form to the data and back, step by step.
Duplicate and incomplete data	Mandatory fields, unique ID and email checks, and validation messages.
Showing user text safely	Escaped every displayed value so quotes and tags appear as plain text.
Mixing logic and storage code	Moved reading and writing into separate data-access methods.

ADDITIONAL SYSTEM DESIGN AND REQUIREMENT ANALYSIS

4.8 Detailed Requirement Analysis

A detailed requirement analysis helps ensure that the Employee Management Analysis application solves the practical problems identified during the system study. The main users are administrators who maintain employee information and viewers who need read-only access. The design must make routine operations straightforward while preventing accidental or unauthorised changes.

The login requirement establishes the user's identity before employee information is displayed. Once authenticated, the application presents options according to the assigned role. Employee creation requires all mandatory fields, a unique employee ID and a valid email address. Updating a record must preserve its identity while allowing authorised details to be changed. Deletion requires an explicit confirmation because it removes information from the active list.

4.8.1 Use of Requirements During Development

The requirements were converted into visible application behaviour. For example, the requirement to search by department was checked by selecting Development and confirming that matching records alone appeared. The requirement for safe deletion was checked by opening the confirmation dialog and verifying that the record was removed only after confirmation. These checks connect the written specification to the output screens shown earlier in Chapter 4.

4.8.2 Acceptance Criteria

A feature is considered acceptable when it performs the expected action, displays a clear result and does not damage unrelated records. A failed validation should keep the user on the form and explain what needs correction. A successful update should show the revised value in the list. A viewer attempting an administrator-only operation should receive an access-denied response and the stored data should remain unchanged.

These criteria are useful because they evaluate more than whether a page opens. They consider correctness, understandable feedback, access control and preservation of data. They also provide a repeatable basis for retesting after future changes.

DATA QUALITY, VALIDATION AND SECURITY

4.9 Data Quality and Validation Strategy

Employee information is useful only when it is complete, consistent and retrievable. The application therefore checks required fields before saving. Employee IDs are numeric and unique, email addresses must follow a basic email pattern, and phone numbers must contain ten digits. The joining date, name and designation are also required. These checks reduce incomplete records and make later searching and reporting more dependable.

Client-side validation gives immediate feedback and reduces unnecessary requests. It highlights the fields that need attention and helps the user correct errors while the form is still open. However, browser validation cannot be treated as the only protection because requests may be sent without using the normal interface. The Java application repeats important checks before changing data.

4.9.1 Duplicate Prevention and Consistency

A duplicate employee ID could cause the wrong record to be edited or make reports inaccurate. The application checks whether an ID already exists before inserting a new record. The database design also defines the employee ID as a primary key and the email as unique. These constraints express the rules close to the data layer and provide a second line of defence when the prototype is later connected to a relational database.

4.9.2 Safe Display of User Input

Names and other text fields may contain punctuation or characters that resemble markup. The implementation escapes displayed values so that such characters are presented as text rather than interpreted as page code. This is important for both correctness and security: user-entered information should not alter the layout or execute as active content when another user opens the record.

4.9.3 Authentication and Authorisation

Authentication checks who is signing in, while authorisation determines which operations that user may perform. The system separates Admin and Viewer permissions. Restricted operations are checked on the server for every request, not merely hidden in the interface. In a production deployment, password storage should use a modern password-hashing function designed for credentials, with unique salts and suitable work factors, rather than relying only on a general-purpose hash.

TESTING METHOD AND QUALITY ASSURANCE

4.10 Expanded Testing Methodology

Testing was organised around the major functions of the application: authentication, employee form validation, record creation, search, update, deletion, reports and role restrictions. Each test begins with a defined input and an expected result. The observed result is then compared with that expectation. This approach makes testing systematic and helps identify which requirement is affected if a problem occurs.

4.10.1 Functional Testing

Functional testing checks whether each feature behaves as specified. A valid login should open the dashboard, while an invalid password should display an error. A valid employee form should create a record that appears in the employee list. Searching for a known department should return matching employees, and searching for an unknown ID should provide a clear no-records message. Editing should persist the revised value, and deletion should require confirmation.

4.10.2 Negative and Boundary Testing

Negative tests deliberately provide invalid or incomplete information. Examples include leaving the employee name empty, entering an invalid email address, supplying a phone number with too few digits and attempting to reuse an employee ID. Boundary testing should also consider very long names, empty search terms, unusual punctuation and dates at the edges of the permitted range. These cases help reveal weaknesses that normal successful inputs may not expose.

4.10.3 Regression Testing

Regression testing is performed after a change to make sure that previously working features still behave correctly. For example, a change to the employee form should be followed by tests for both adding and editing records, because the same form is shared. A change to role handling should be checked with both Admin and Viewer accounts. Re-running the existing test cases provides a practical baseline for future development.

4.10.4 Test Evidence and Limitations

The screenshots in Chapter 4 provide visual evidence of representative application states. The recorded test table reports twelve planned cases as passed. The results support the behaviour demonstrated by those cases, but they do not prove that every possible input, browser, load level or security threat has been tested. Broader testing would be required before the application was used for real organisational records.

DEPLOYMENT, MAINTENANCE AND OPERATION

4.11 Deployment Considerations

The prototype starts a Java HTTP server on port 8080 and serves pages under the EmployeeManagement path. A user accesses the application through a web browser on a machine that can reach the server. Before deployment, the Java Development Kit version, file permissions, data-file location and port availability should be checked. The process should be started with an account that has only the permissions required to run the application and write its data file.

4.11.1 Data Storage and Backup

The current prototype uses a CSV file for persistence. This is convenient for demonstrating the basic workflow and keeping the setup simple, but it has limitations. Concurrent updates, complex queries, relationships and recovery from partial writes are more difficult to manage than with a database. A scheduled backup should copy the data file to a protected location, and a restoration test should be carried out to confirm that a backup can actually be used.

The proposed relational model provides a migration path. Department details can be stored once in a Department table, employee rows can refer to departments through a foreign key, and user accounts can be stored separately. JDBC and prepared statements can then be used for database access. The data-access methods should remain separate from the presentation code so that storage can be replaced without rewriting every page.

4.11.2 Maintenance Activities

Routine maintenance includes reviewing error messages, checking whether the application starts correctly, backing up data, updating the Java runtime and reviewing user access. Changes should be tested in a separate environment before being applied to the running version. A short change log should record what was modified, why it was needed and which tests were repeated.

4.11.3 Operational Limitations

The current implementation is an educational prototype rather than a fully hardened enterprise system. A production version would need secure transport, robust session expiry, stronger password hashing, audit logs, database transactions, centralised configuration and controlled recovery procedures. These improvements would reduce operational risk and make the system more suitable for handling confidential employee information.

USER EXPERIENCE, REPORTING AND FUTURE WORK

4.12 Usability and User Experience

A useful employee-management interface should help users understand where they are, what actions are available and whether an operation has succeeded. Consistent navigation, descriptive field labels, visible required-field indicators and clear feedback reduce confusion. Search and department filters should be easy to locate because users may consult the employee list frequently. Destructive actions such as deletion should be visually distinct and require confirmation.

The Admin and Viewer interfaces should remain consistent in their read-only areas so that users can find information in the same way. The difference should be limited to permissions and relevant actions. This avoids creating two unrelated experiences while still protecting employee records from unauthorised modification. Responsive layouts, keyboard navigation, sufficient contrast and clear focus indicators would improve accessibility across different devices and users.

4.12.1 Reporting and Decision Support

Department-wise reporting provides a quick overview of how employee records are distributed. Counts of total, active and inactive employees can help administrators identify incomplete status information and understand the current composition of the organisation. A chart makes differences between departments easier to notice, while a table preserves exact counts. Reports should be generated from the same underlying records as the employee list so that the displayed totals remain consistent.

Before a report is used for a management decision, its filters and definitions should be clear. For example, an inactive employee should not be counted as active, and an employee should belong to a valid department. The report should also handle departments with no current employees and should present an understandable message when no data is available.

4.12.2 Future Development Roadmap

The next development stage is to migrate CSV storage to a relational database and use JDBC with prepared statements. A subsequent stage could introduce Spring Boot and REST endpoints, separating the front end from the service layer. Further improvements could include attendance, leave and payroll modules, report export to PDF or Excel, email notifications and account-management features.

A final stage should focus on deployment readiness: automated unit and integration tests, accessibility checks, responsive design, audit logging, backup-and-restore verification and a security review. These steps would build on the working prototype while preserving its main objective: providing a centralised, validated and role-controlled way to manage employee information.

The expanded discussion complements the architecture, implementation, output screens and test cases already documented. It clarifies how the current prototype can be maintained and how it could be developed further without claiming that proposed features have already been implemented.

CHAPTER 5: CONCLUSION

5.1 Conclusion

The internship at Besant Technologies gave practical experience of how the front end, the Java back end and the data layer of a web application fit together. The project, Employee Management Analysis, was built and tested as a working application with login and roles, validated forms, search, CRUD operations and a department-wise report. All twelve planned test cases passed, and the screens in Chapter 4 come from the running application.

Besides technical skills in Java, HTML, CSS, JavaScript and SQL design, the internship improved problem solving, planning within a fixed one-month period and the habit of documenting and testing work. It forms a good base for larger projects and for a career in web application development.

5.2 Future Enhancements

- Replace the file storage with a relational database using JDBC and the SQL design of Section 3.3.
- Use a framework such as Spring Boot to build a REST API for the back end.
- Add attendance, leave and payroll modules.
- Export reports to PDF and Excel and add email notifications.
- Make the interface fully responsive for phones and tablets.
- Add password change and account management for users.

REFERENCES

- Oracle Corporation, Java Documentation – https://docs.oracle.com/en/java/
- Oracle Corporation, JDBC Basics Tutorial – https://docs.oracle.com/javase/tutorial/jdbc/
- MDN Web Docs, HTML, CSS and JavaScript – https://developer.mozilla.org/
- OWASP Cheat Sheet Series – https://cheatsheetseries.owasp.org/
- Besant Technologies – https://www.besanttechnologies.com/